Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Projektowanie Efektywnych Algorytmów

Sprawozdanie z zadania nr 1

Daniel Gościński 280878

Prowadzący:
dr inż. Jarosław Mierzwa

Spis treści

1. Wstęp teoretyczny 3
2. Opis Algorytmów 3
3. Plan eksperymentów 4
   3.1. Badanie wpływu rozmiaru problemu na długość wykonywania programu 4
   3.2. Badanie średniego błędu względnego algorytmów aproksymacyjnych 5
4. Wyniki eksperymentów 6
   4.1. Badanie wpływu rozmiaru problemu na długość wykonywania programu 6
   4.2. Badanie średniego błędu względnego algorytmów aproksymacyjnych 7
5. Wnioski 8
6. Literatura 10

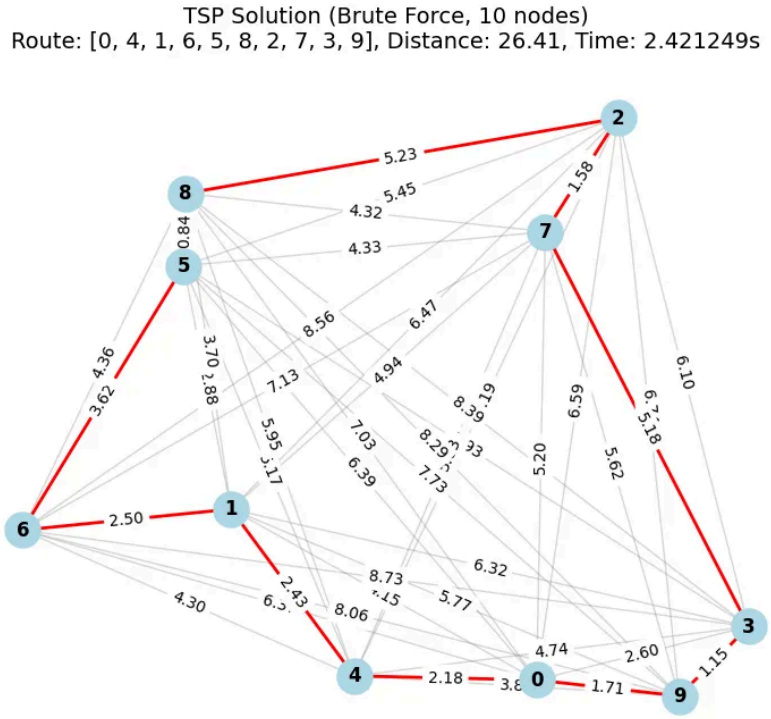
1. Wstęp teoretyczny

Problem komiwojażera to jeden z najstarszych i dobrze znanych problemów klasy NP-trudnych. Oznacza to, że nie udało się uzyskać dla niego algorytmu, który potrafiłby znaleźć optymalne rozwiązanie w czasie wielomianowym.

Zadanie te polega na znalezieniu najmniejszego cyklu Hamiltona w grafie ważonym[1]. Wyróżniamy dwa rodzaje problemu:

- Symetryczny: odległości między dwoma miastami są identyczne
- Asymetryczny: odległości między dwoma miastami nie są identyczne (Droga A-B ma inną wagę niż droga B-A)

Na potrzeby zadania rozważany jest wariant asymetryczny (**Asymetric Travelling Salesman Problem**).



Zdjęcie 1: Przykładowa wizualizacja problemu ATSP

https://lopezyse.medium.com/graphs-solving-the-travelling-salesperson-problem-tsp-in-python-54ec2b315977

2. Opis Algorytmów

Na potrzeby zadania analizowane będą cztery algorytmy rozwiązujące problem.

Pierwszy algorytm to przegląd zupełny (ang. Bruteforce), z którego uzyskiwane jest dokładne rozwiązanie. Dzieje się to jednak kosztem czasu wykonania algorytmu. Posiada on złożoność czasową **O(N!)**, gdyż dla N-wierzchołkowego grafu przegląd zupełny wykona **(N-1)!** permutacji, co powoduje długi czas wykonywania algorytmu. Dodatkowo czas ten będzie rósł z dużym przyśpieszeniem. Złożoność pamięciowa

tego algorytmu wynosi $\mathbf{O}(N^2)$, gdyż graf przechowywany jest w postaci macierzy sąsiedztwa.

Pozostałe algorytmy są algorytmami aproksymacyjnymi. Oznacza to, że wynik nie jest gwarantowany do bycia optymalnym.

Algorytm Najbliższego Sąsiada (ang. Nearest Neighbor) jest jednym z takich algorytmów. Polega on na zachłannym wybieraniu najkrótszej możliwej ścieżki w danym kroku (stąd nazwa najbliższego sąsiada). Algorytm ten posiada złożoność obliczeniową $\mathbf{O}(N^2)$. Jednym z problemów w tym algorytmie jest rozstrzygnięcie przypadku, w którym wielu sąsiadów posiada taką samą odległość od miasta, w którym jesteśmy. W przypadku tego projektu algorytm posiada rozwiązanie rozstrzygania tego problemu poprzez sprawdzenie pełnej drogi dla każdego z sąsiadów, a następnie wybraniu najkrótszej ścieżki.

Następnym algorytmem jest Powtarzalny Najbliższy Sąsiad (ang. Repetitive Nearest Neighbor). Jest to rozwinięcie algorytmu NN, w którym dla każdego wierzchołka grafu wykonujemy algorytm Najbliższego Sąsiada. Ze względu na większą ilość wykonań, zakłada to zwiększenie dokładności algorytmu, co zostanie poddane analizie. Ceną zwiększenia precyzji jest czas wykonywania, który w tym przypadku jest $\mathbf{O}(N^3)$, ponieważ musimy wykonać algorytm NN dla każdego (N) wierzchołka.

Ostatnim algorytmem jest Algorytm Losowy. Jego działanie jest proste w opisie, ponieważ polega on na wygenerowaniu losowej permutacji, a następnie zliczeniu wartości dystansów pomiędzy wierzchołkami. Posiada on złożoność obliczeniową **O(N)**, jednak z natury swojej losowości jest on mało dokładny.

3. Plan eksperymentów

W celach badawczych algorytmów został utworzony program, pozwalający na wykonywanie czterech algorytmów ATSP na grafach wczytanych z pliku, lub wygenerowanych losowo. Zakłada się, że grafy zawsze są przedstawione w postaci macierzy sąsiedztw, gdzie na przekątnej macierzy pojawia się wartość -1, a w pozostałych komórkach waga ścieżki pomiędzy wybraną dwójką krawędzi. Z wykorzystaniem utworzonego programu wykonane zostały dwa badania:

3.1. Badanie wpływu rozmiaru problemu na długość wykonywania programu

W tym badaniu zmierzono czas wykonywania się przeglądu zupełnego dla 7 wybranych rozmiarów macierzy sąsiedztw. Z powodu złożoności obliczeniowej algorytmu, zdecydowano się na rozmiary {8,9,10,11,12,13,14}. Badanie przeprowadzono w następujący sposób:

1. Uruchomiono odpowiedni podprogram (numer 5), jako początkową liczbę miast podano wartość 8, oraz wartość minimalną krawędzi 10, a maksymalną 100.
2. Program generuje losowy graf z ustaloną ilością miast, oraz zakresem wag tras
3. Program mierzy czas wykonania algorytmu
4. Program zwraca czas wykonania algorytmu dla danego rozmiaru problemu

Punkty 2-4 powtarzane są siedem razy, za każdym razem zwiększając rozmiar N o jeden. Dane losowe generowane są z wykorzystaniem generatora **uniform_int_distribution**, aby zapewnić losowość generowanych wartości.

3.2. Badanie średniego błędu względnego algorytmów aproksymacyjnych

W tym badaniu zbadany został błąd względny algorytmów, które nie gwarantują uzyskania dokładnego, optymalnego rozwiązania. W tym celu wybrano 5 rozmiarów N {10,11,12,13,14}, dla których zbadano różnicę uzyskanego przez algorytm wyniku wagi ścieżki, od wagi dokładnej uzyskanej przy pomocy metody BruteForce. Badanie przeprowadzono w następujący sposób:

1. Uruchomiono odpowiedni podprogram (numer 6), jako początkową liczbę miast podano wartość 10
2. Program generuje losowy graf o wielkości aktualnego N i wagami krawędzi z przedziału <1;100>
3. Program uruchamia algorytm BruteForce, z którego uzyskuje wartość optymalną ścieżki
4. Program uruchamia każdy algorytm aproksymacyjny, z którego uzyskuje wartość ścieżki uzyskanej danym algorytmem
5. Program oblicza sumę błędów względnych dodając wynik następującego wzoru

$$\frac{\text{kosztAlgorytmuAproksymacyjnego} - \text{kosztAlgorytmuBruteForce}}{\text{kosztAlgorytmuBruteForce}} * 100\%$$

1. Na koniec program dzieli sumę błędów przez ilość wykonanych iteracji
2. Program zwraca wartość średnich błędów względnych algorytmów

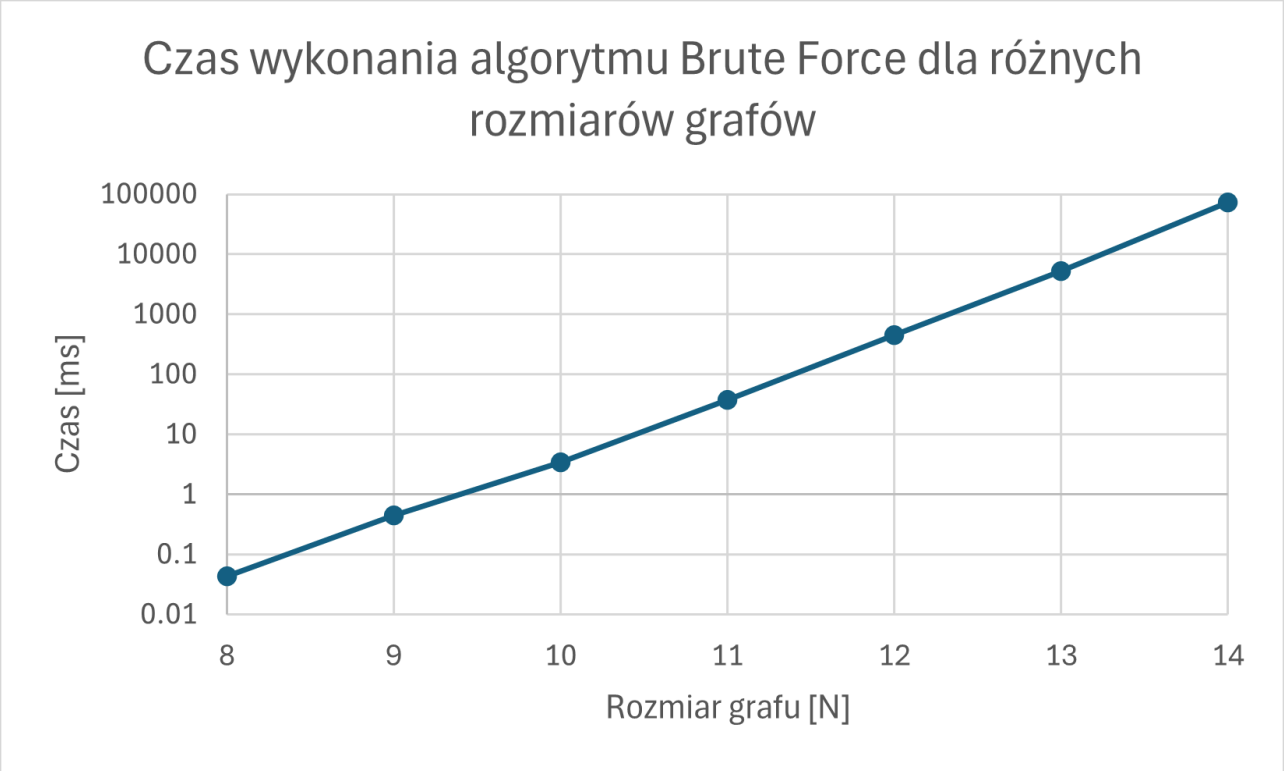
Punkty 2-5 wykonywane są dla każdej wartości N po 100 razy w celu uśrednienia wyników.

4. Wyniki eksperymentów

4.1. Badanie wpływu rozmiaru problemu na długość wykonywania programu

Tabela 1: Czas wykonywania algorytmu BruteForce, w zależności od rozmiaru N

Ilość wierzchołków N	Czas [ms]
8	0.043
9	0.451
10	3.413
11	37.17
12	453.238
13	5209.238
14	72 346.566



Wykres 2: Czas wykonywania algorytmu Brute-Force w zależności od rozmiaru problemu (N). Zastosowano logarytmiczną skalę osi Y ze względu na silniową złożoność obliczeniową $O(N!)$ badanej metody

Analizując uzyskane wyniki możemy potwierdzić złożoność obliczeniową $O(N!)$ algorytmu przeglądu zupełnego. Ponieważ dane rosły w bardzo dużym tempie, w wyniku czego czasy dla większych rozmiarów N całkowicie zasłaniały wartości mniejszych rozmiarów zastosowano skalę logarytmiczną, w wyniku czego wykres przyjął postać zbliżony do prostej. Oznacza to, że czas wykonania dla każdego kolejnego N zwiększał się w stosunku do poprzedniego rozmiaru N-krotnie.

Badania te pozwalają nam zrozumieć, że uzyskanie optymalnego rozwiązania zaczyna być bardzo czasochłonne dla grafów o rozmiarach około 15 lub więcej. W takich przypadkach należy starać się stosować inne algorytmy, które będą próbowały uzyskać wynik w znacznie krótszym czasie.

4.2. Badanie średniego błędu względnego algorytmów aproksymacyjnych

Tabela 2: Średni błąd względny dla algorytmów NN

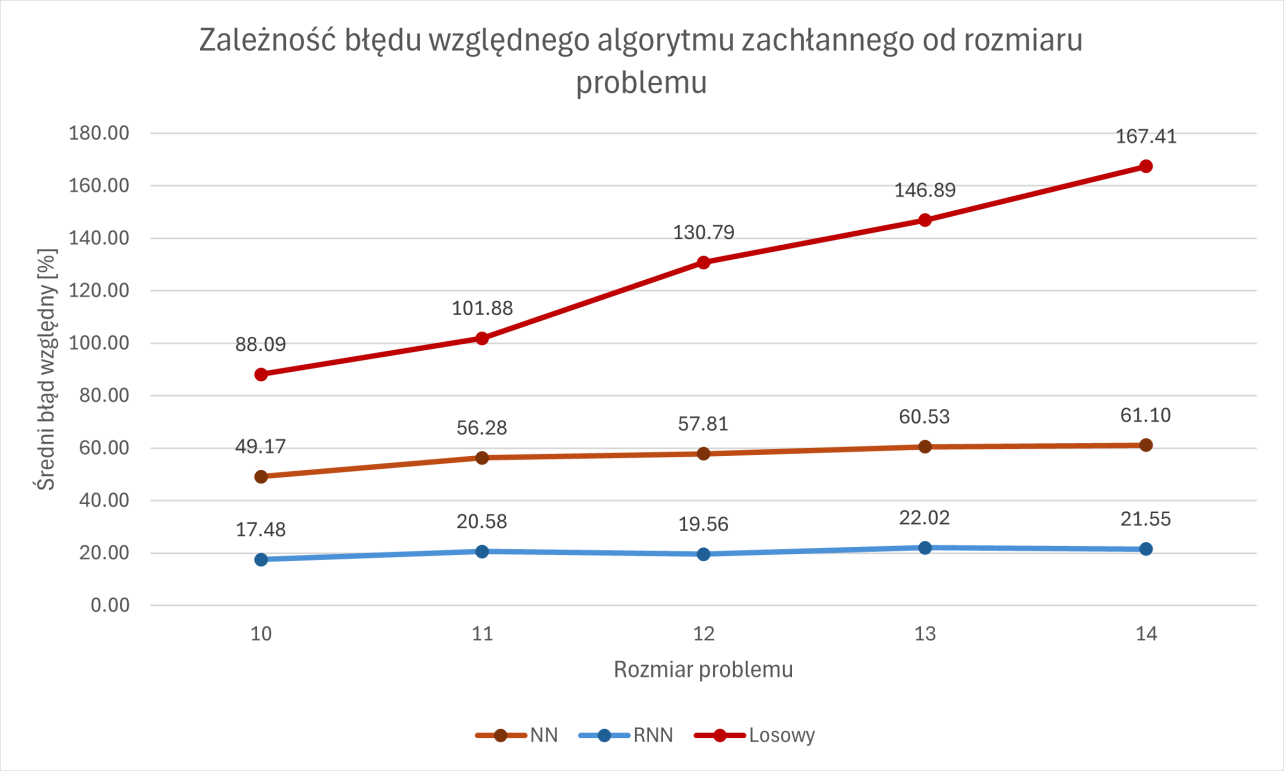
Ilość wierzchołków N	Średni błąd względny [%]
10	49.17
11	56.28
12	57.81
13	60.53
14	61.10

Tabela 3: Średni błąd względny dla algorytmu RNN

Ilość wierzchołków N	Średni błąd względny [%]
10	17.48
11	20.58
12	19.56
13	22.02
14	21.55

Tabela 4: Średni błąd względny dla algorytmu losowego

Ilość wierzchołków N	Średni błąd względny [%]
10	88.09
11	101.88
12	130.79
13	146.89
14	167.41



Wykres 3: Średni błąd względny algorytmów aproksymacyjnych dla wybranych rozmiarów N.

Analizując uzyskane dane można od razu zauważyć przewagę metod heurystycznych nad przeszukiwaniem losowym. W porównaniu do pozostałych algorytmów błąd względny dla Random Search stale rośnie w o wiele większym tempie od pozostałych algorytmów. Wynika to z o wiele mniejszego prawdopodobieństwa na uzyskanie zadowalającej ścieżki.

Kolejnym wnioskiem jaki można zauważyć to wpływ wyboru wierzchołka na metodę Najbliższego Sąsiada. Algorytm RNN stale uzyskuje mniejszy błąd względem NN, pomimo, że w założeniu wykonują ten sam algorytm. Udowadnia to wpływ na jakość rozwiązania wybór poczatkowego miasta. Z tego też powodu algorytm RNN uzyskuje lepsze wyniki.

Możemy zauważyć również, że pomimo wzrostu liczby miast w problemie, metody heurystyczne utrzymują stabilność w zakresie odchyłu uzyskanych wyników od wyniku optymalnego. Oznacza to, że algorytmy te lepiej sobie radzą z intensywnym zwiększaniem się ilości możliwych permutacji w problemie.

5. Wnioski

Wykonane badania Asymetrycznego Problemu Komiwojażera pozwoliły na zauważenie kluczowych kwestii w zakresie czasu działania algorytmu dokładnego w problemie klasy NP-trudny, oraz na możliwe metody uzyskiwania przybliżonego rozwiązania problemu.

Na podstawie wykonanych badań można by również zastanowić się, jak na algorytmy wpływały by takie parametry jak gęstość grafu, istnienie większej ilości optymalnych ścieżek czy zakres wag krawędzi.

Dla metody BF, zmniejszenie gęstości grafu spowodowałoby zmniejszenie ilości wykonywanych obliczeń, przy założeniu, że jeżeli w danej permutacji pojawia się ścieżka, która nie istnieje, to cała permutacja jest od razu porzucana. Mimo to w dalszym ciągu w przypadku pesymistycznym (graf pełny) czas wykonywania będzie zgodny z tymi uzyskanymi w badaniach. Istnienie większej ilości ścieżek o optymalnym koszcie nie zmienia wiele w czasie działania algorytmu - w dalszym ciągu musi wykonać się wszystkie N! możliwości. Zakres wag nie zmieniałby nic w algorytmie.

Dla NN i RNN gęstość grafu algorytm utworzyłaby mniejszą ilość sąsiadów do sprawdzania na każdym kroku, jednak zwiększa prawdopodobieństwo znalezienie ślepego zaułka, w wyniku którego algorytm musiałby wykonywać backtracking. W takich przypadkach może okazać się, że algorytmy nie będę w stanie w ogóle znaleźć cyklu Hamiltona. Ze względu na swoją naturę zachłanną większa ilość optymalnych rozwiązań nie wpływałaby na algorytmy, ponieważ ich działania szukania optimum lokalnego nie zmienia liczby operacji do wykonania. Zmniejszenie zakresu wag może negatywnie wpłynąć na algorytmy, ponieważ może pojawić się więcej sąsiadów o tych samych wagach.

Algorytm losowy nie przejmuje się za bardzo zmianami w porównaniu do poprzednich algorytmów. Gęstość grafu czy zakres wag nie zmienia działania algorytmu, który i tak losowo wybiera permutację do sprawdzenia. Istnienie większej ilości optymalnych ścieżek zwiększa prawdopodobieństwo, że wylosowana permutacja będzie tą optymalną, jednak w dalszym ciągu przy ilości możliwych kombinacji jest to nadal bardzo nieefektywna metoda.

Przeprowadzone badania potwierdziły złożoność obliczeniową przeglądu zupełnego, którym jest O(N!). Ze względu na ilość permutacji, jaka pojawia się wraz z zwiększeniem wartości N eliminuje możliwość wykorzystywania metody dokładnej do uzyskania wyniku.

Równie zawodną metodą poszukiwania wyniku jest metoda losowa. Błąd względny rozwiązania rośnie najszybciej z badanych algorytmów, w wyniku czego w przestrzeni rosnącej silniowo nie warto jest próbować ślepego wyboru rozwiązania.

Metody zachłanne dobrze radzą sobie z zadaniem znalezienia ścieżki bliskiej optymalnej. Wiedzę tą można wykorzystać, przykładowo do utworzenia ograniczeń dla przeszukiwań rozwiązań, w celu zmniejszenia czasu znajdywania wyniku dokładnego.

6. Literatura

[1] S. Lin i B. W. Kernighan, „An Effective Heuristic Algorithm for the Traveling--Salesman Problem”, *Operations Research*, t. 21, nr 2, s. 498–516, 1973, Dostęp: 25 marca 2026. [Online]. Dostępne na: http://www.jstor.org/stable/169020